P.Vishwa Nandan(192473015)

Q1 – Handling Failure in a Bank ATM Simulation

Marks: 6

1. Introduction

An ATM (Automated Teller Machine) allows users to withdraw money from their bank accounts.

During a withdrawal, different errors may occur. For example, users may enter letters instead of numbers, enter negative amounts, request more money than their account balance or use a missing card-reader file.

Java provides exception handling mechanisms to handle these failures without stopping the entire program unexpectedly.

The main exception-handling keywords are:

- try – Contains code that may generate an exception.
- catch – Handles an exception.
- finally – Executes cleanup code whether an exception occurs or not.
- throw – Explicitly generates an exception.
- throws – Declares exceptions that a method may generate.

2. Identifying ATM Failure Points

The following failures can occur during ATM withdrawal.

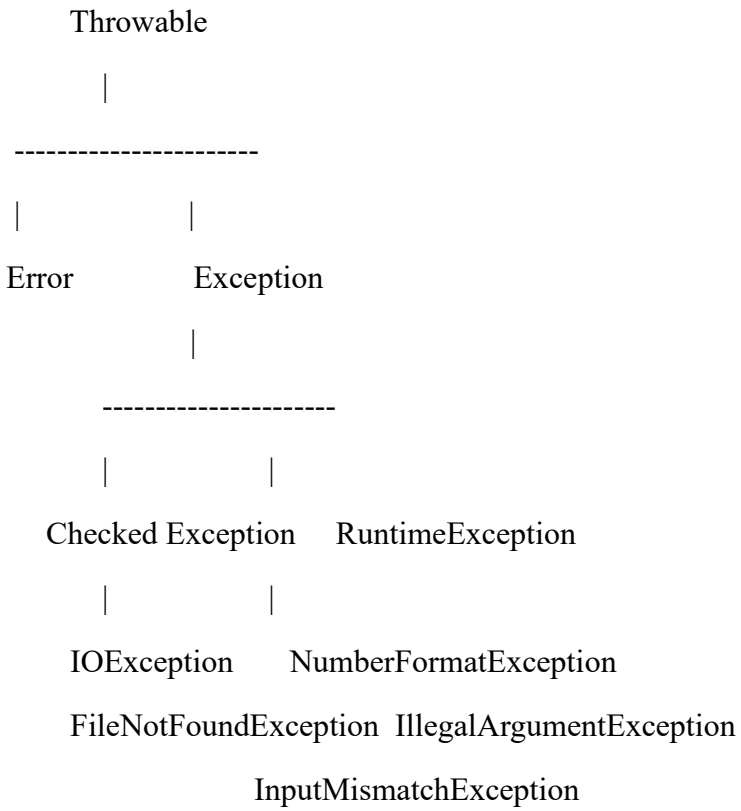
1. The user enters letters instead of a numeric account number.
2. The user enters letters instead of a withdrawal amount.
3. The user enters a negative withdrawal amount.
4. The user requests an amount greater than the available balance.
5. The card-reader file is missing.
6. The application does not have permission to access the card-reader file.
7. An unexpected system error occurs.

These failures must be identified and handled using suitable Java exceptions.

3. Throwable Hierarchy

In Java, Throwable is the root class of all errors and exceptions.

The relevant hierarchy is:



Explanation:

1. Error represents serious problems related to the Java Virtual Machine, such as OutOfMemoryError.
2. Checked exceptions are checked by the compiler and must be handled or declared.
3. Unchecked exceptions occur during program execution and do not require mandatory handling.

4. Failure Classification Table

Failure Point	**Exception Type**	**Classification**	**Handling**
Letters instead of account number	NumberFormatException	Unchecked	Catch block
Letters instead of withdrawal amount	NumberFormatException	Unchecked	Catch block
Negative withdrawal amount	IllegalArgumentException	Unchecked	Catch block
Withdrawal greater than balance	InsufficientFundsException	Checked	Catch block

Missing card-reader file	NoSuchFileException	Checked	IOException catch block
No permission to access file	SecurityException	Unchecked	Multi-catch block
Serious JVM memory failure	OutOfMemoryError	Error	Normally not handled by application code

InsufficientFundsException is a user-defined checked exception created for the ATM application.

5. Exception Handling Using try, catch and finally

The ATM uses a try block to perform withdrawal operations.

Different catch blocks handle different failures.

A multi-catch block can handle IOException and SecurityException because they are unrelated exception types.

The finally block closes resources and displays a session completion message.

6. Expected Output and Testing

The following test cases demonstrate different exception-handling paths.

Test Case 1 – Valid Withdrawal

Input:

ATM WITHDRAWAL SYSTEM

Enter account number: 123456

Enter withdrawal amount: 1000

Output:

Withdrawal successful

Withdrawn Amount: Rs. 1000

Remaining Balance: Rs. 4000

ATM session closed.

Test Case 2 – Invalid Account Number

Input:

Enter account number: ABC123

Output:

Invalid input: Please enter numeric values.

ATM session closed.

Exception: NumberFormatException.

Test Case 3 – Negative Withdrawal Amount

Input:

Enter account number: 123456

Enter withdrawal amount: -500

Output:

Invalid withdrawal: Withdrawal amount must be positive

ATM session closed.

Exception: IllegalArgumentException.

Test Case 4 – Withdrawal Greater Than Balance

Input:

Enter account number: 123456

Enter withdrawal amount: 6000

Output:

Transaction declined: Insufficient account balance

ATM session closed.

Exception: InsufficientFundsException.

Test Case 5 – Missing Card-Reader File

Input:

Enter account number: 123456

Enter withdrawal amount: 1000

When card-reader.txt is missing, the output is similar to:

Card-reader error: card-reader.txt

ATM session closed.

Exception: NoSuchFileException, handled through its parent class IOException.

Test Case 6 – Card Reader Not Ready

If the card-reader file does not contain READY, the output is:

ATM error: Card reader is not ready

ATM session closed.

Exception: IllegalStateException.

Test Results Table

Test	Input Condition	Exception	Finally Executed
1	Valid withdrawal	No exception	Yes
2	Letters instead of account number	NumberFormatException	Yes
3	Negative amount	IllegalArgumentException	Yes
4	Amount greater than balance	InsufficientFundsException	Yes
5	Missing card-reader file	IOException	Yes
6	Card reader not ready	IllegalStateException	Yes

7. Justification for the Order of Catch Blocks

The catch blocks must be arranged carefully.

Java requires more specific exceptions to be caught before their broader parent exceptions.

The correct order is:

1. NumberFormatException
2. IllegalArgumentException
3. InsufficientFundsException
4. IllegalStateException
5. IOException | SecurityException

NumberFormatException is a subclass of IllegalArgumentException. Therefore, NumberFormatException must be caught before IllegalArgumentException.

This ensures that the correct exception handler is executed.

If a parent exception is caught first, the compiler may report an unreachable catch block.

8. Purpose of the finally Block

The finally block executes whether the try block completes normally or an exception occurs.

In the ATM program, it performs the following operations:

1. Closes the card-reader file if it was successfully opened.

2. Closes the Scanner.
3. Displays the ATM session completion message.

The finally block ensures that cleanup operations are attempted even when an exception occurs.

It executes in normal exception-handling flow, although abnormal JVM termination or similar failures can prevent it from executing.

STUDENT A – Exception Design

9. Failure Classification and Catch Order

The ATM uses different exception types for different failures.

Checked exceptions, such as IOException and InsufficientFundsException, must be handled or declared.

Unchecked exceptions, such as NumberFormatException and IllegalArgumentException, do not require mandatory handling by the compiler.

Specific catch blocks improve error identification and make debugging easier.

10. Explanation of finally Block

The finally block guarantees that cleanup code is executed after the try-catch structure.

Even when the withdrawal operation fails, the program attempts to close the card-reader file and Scanner.

This prevents unnecessary resource usage and improves application reliability.

STUDENT B – Testing and Tracing

11. Stack Trace Demonstration

A stack trace shows the sequence of method calls that caused an exception.

The following program generates an uncaught exception.

File Name: StackTraceDemo.java

```
public class StackTraceDemo {


   static void readCard() {


      throw new IllegalStateException(
            "ATM hardware failure"
      );
```

```
    }

    static void processWithdrawal() {

        readCard();
    }

    public static void main(String[] args) {

        processWithdrawal();
    }
}
```

Representative Stack Trace

Exception in thread "main" java.lang.IllegalStateException:

ATM hardware failure

at StackTraceDemo.readCard(StackTraceDemo.java:5)

at StackTraceDemo.processWithdrawal(StackTraceDemo.java:12)

at StackTraceDemo.main(StackTraceDemo.java:16)

The exact line numbers depend on the saved source file.

Explanation

Line 1: Shows the exception type and the thread in which it occurred.

Line 2: Displays the exception message.

Line 3: Shows where the exception was generated inside readCard().

Line 4: Shows that processWithdrawal() called readCard().

Line 5: Shows that main() called processWithdrawal().

The stack trace helps programmers identify the source of an error and follow the sequence of method calls.

12. What Happens if a Broader Catch Block Comes First?

Consider:

```
try {

    int amount = Integer.parseInt("ABC");

}

catch (Exception e) {

    System.out.println("Exception occurred");
}

catch (NumberFormatException e) {

    System.out.println("Invalid number");
}
```

This program produces a compilation error.

Reason:

NumberFormatException is already covered by the earlier Exception catch block.

Therefore, the second catch block becomes unreachable.

The correct approach is:

```
try {

    int amount = Integer.parseInt("ABC");

}

catch (NumberFormatException e) {
```

```
    System.out.println("Invalid number");
}

catch (Exception e) {

    System.out.println("Other exception occurred");
}
```

Specific exceptions must be handled before general exceptions.

JOINT DESIGN QUESTION

13. Should Every Exception Be Handled Using catch(Exception e)?

Using only one catch(Exception e) block is generally not recommended for the ATM application.

Different exceptions represent different failures.

For example:

- NumberFormatException indicates invalid numeric input.
- InsufficientFundsException indicates insufficient account balance.
- IOException indicates a file-access problem.

If all exceptions are handled using one general catch block, the program may lose important information about the actual failure.

When Is catch(Exception e) Acceptable?

It can be used at a top-level application boundary as a final safety net for unexpected exceptions, provided that the error is properly logged and the application responds safely.

However, specific catch blocks should be used when different failures require different handling.

Therefore, using specific exception handlers improves readability, debugging and error management.

ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

14. Alternative Approach Without Exceptions

Instead of using exceptions, the ATM can validate all inputs before performing the withdrawal.

Scanner provides the hasNextInt() method to check whether the next input is an integer.

The program can return error codes instead of throwing exceptions.

Example:

```
import java.util.Scanner;

class BankAccount {

    int balance = 5000;

    // Returns 0 for success
    // Returns -1 for invalid amount
    // Returns -2 for insufficient balance

    int withdraw(int amount) {

        if (amount <= 0) {
            return -1;
        }

        if (amount > balance) {
            return -2;
        }

        balance = balance - amount;

        return 0;
    }
}

public class Main {
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    BankAccount account = new BankAccount();

    System.out.println("Enter withdrawal amount:");

    if (!sc.hasNextInt()) {

        System.out.println("Invalid numeric input");

        sc.next();

    } else {

        int amount = sc.nextInt();

        int result = account.withdraw(amount);

        if (result == 0) {
            System.out.println("Withdrawal successful");
        }

        else if (result == -1) {
            System.out.println("Invalid amount");
        }
```

```
            else if (result == -2) {
                System.out.println("Insufficient balance");
            }
        }

        sc.close();
    }
}
```

Expected Output for Valid Input

Enter withdrawal amount:

1000

Withdrawal successful

Expected Output for Invalid Input

Enter withdrawal amount:

ABC

Invalid numeric input

Expected Output for Insufficient Balance

Enter withdrawal amount:

6000

Insufficient balance

15. Comparison Between Exception Handling and Error Codes

Exception Handling:

- Uses try, catch, finally, throw and throws.
- Separates normal program execution from error handling.
- Provides detailed information about failures.
- Stack traces help identify the source of errors.
- Creating and throwing exceptions involves additional runtime overhead.

Error-Code Approach:

- Uses predefined error codes such as -1 and -2.

- Requires explicit validation checks.
- Does not automatically provide stack traces.
- May make the code less readable when many error conditions exist.
- Avoids exception stack-trace creation when returning error codes.

16. Complexity Analysis

Let d represent the number of method calls in the exception stack trace.

A. Exception Handling

- Normal validation time: O(1) for a fixed number of checks.
- Creating an exception and capturing its stack trace: approximately O(d).
- Additional space for the exception and stack trace: O(d).

Throwing an exception can involve additional processing because Java creates an exception object and captures information about the method-call stack.

B. Error-Code Approach

- Validation time: O(1) for a fixed number of checks.
- Returning an error code: O(1).
- Additional space: O(1).

Returning an error code does not require creating a stack trace.

Comparison Table

Feature	**Exception Handling**	**Error-Code Approach**
Validation	try-catch and validation	Explicit conditions
Execution cost	Higher when exceptions are thrown	Lower
Error information	Detailed exception message and stack trace	Numeric error code
Readability	Better for complex failure handling	Can become difficult with many checks
Additional space	O(d) for stack trace	O(1)
Best use	Unexpected and exceptional failures	Expected validation failures

17. Conclusion

Java exception handling helps the ATM application manage invalid inputs, negative withdrawal amounts, insufficient balances and file-access problems. Checked and unchecked exceptions provide different ways of handling failures. The try, catch and finally blocks improve reliability by handling errors and closing resources. Although error codes are faster for expected validation failures, exception handling is more suitable when detailed error information and clear failure propagation are required.